

Lecture 4 - Software and Operating Systems

Overview

This lecture covers the types of **software**, including **systems software** and **application software**, the role of **operating systems**, and the **software development process** with common models and methods.

Topics

- | | |
|---|--|
| 1. Computer Software | 5. Operating Systems |
| 2. Systems Software | 6. Software Development |
| 3. Application Software | 7. Software Development Models |
| 4. Major Functions of Systems Software | 8. Preferred Models in Digital Transformation Era: Agile and DevOps |

Objectives / Outcome

1. Differentiate between different types of **computer software**.
2. Understand the role and functions of **systems software**.
3. Explain the purpose and types of **application software**.
4. Describe the major functions of an **operating system**.
5. Understand the **software development process** and common development models.

Computer Software

Software refers to a set of instructions, data, or programs used to operate computers and perform specific tasks.

Types of Software

- **Systems Software**
- **Application Software**

Systems Software

Systems software includes the programs that manage the hardware components and the overall operations of a computer system.

Components

- **Operating Systems (OS):** Manage computer hardware, software resources, and provide common services.
- **Device Drivers:** Allow the OS to communicate with hardware devices.
- **Utility Programs:** Tools that help analyze, configure, optimize, or maintain the computer.

Examples

- Windows OS
- Linux OS
- macOS

Application Software

Application software is designed to help users perform specific tasks or activities.

Types of Application Software

- **Productivity Software:** Microsoft Office (Word, Excel, PowerPoint)
- **Media Software:** Adobe Photoshop, VLC Media Player
- **Educational Software:** Khan Academy, Duolingo
- **Business Software:** SAP, Salesforce
- **Web Browsers:** Google Chrome, Mozilla Firefox

Major Functions of Systems Software

- **Process Management:** Handles process scheduling and execution.
- **Memory Management:** Allocates and deallocates memory for programs.
- **File System Management:** Manages file storage, retrieval, and permissions.
- **Device Management:** Controls hardware devices using drivers.
- **Security and Access Control:** Protects data and resources through authentication and authorization.

Operating Systems

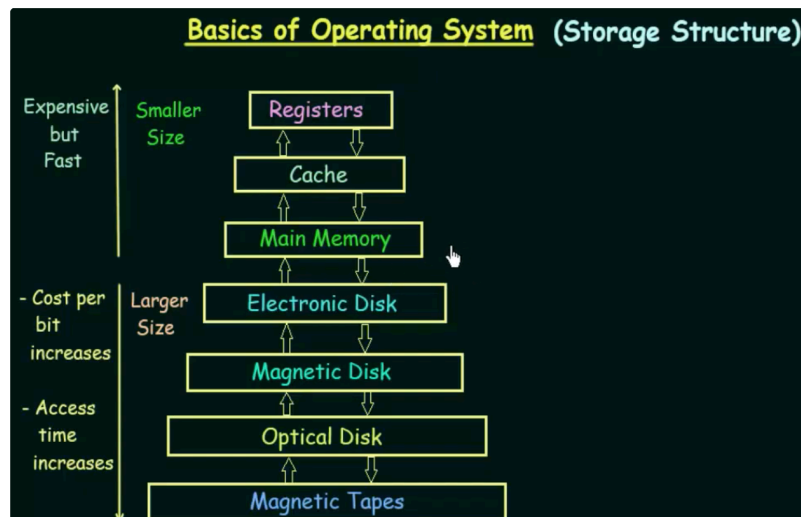
Definition

An **Operating System (OS)** is a type of systems software that manages hardware and software resources on a computer.

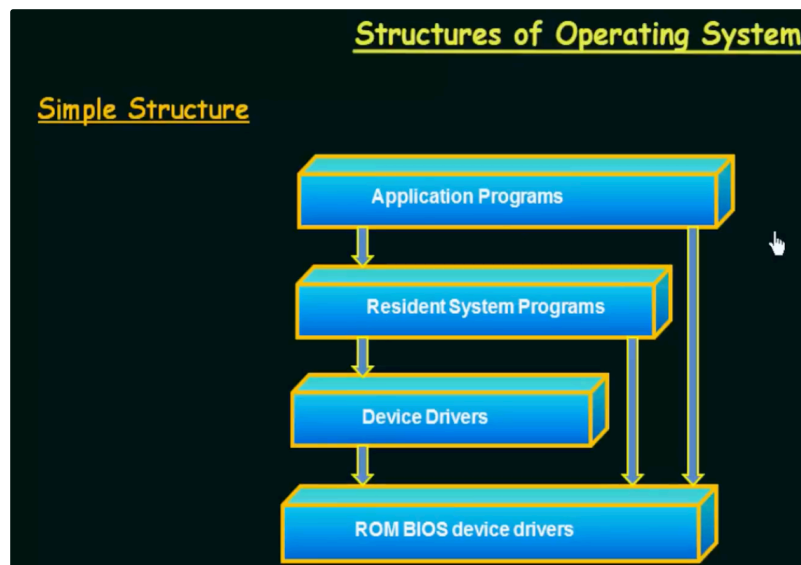
Types of Operating Systems:

- **Batch Operating Systems**
 - Execute batches of jobs without user interaction.
- **Time-Sharing Operating Systems**
 - Allow multiple users to use the system simultaneously by sharing time on the CPU.
- **Distributed Operating Systems**
 - Manage a group of independent computers and make them appear to be a single computer.
- **Real-Time Operating Systems**
 - Provide immediate processing and responses to inputs.

Basics of OSs

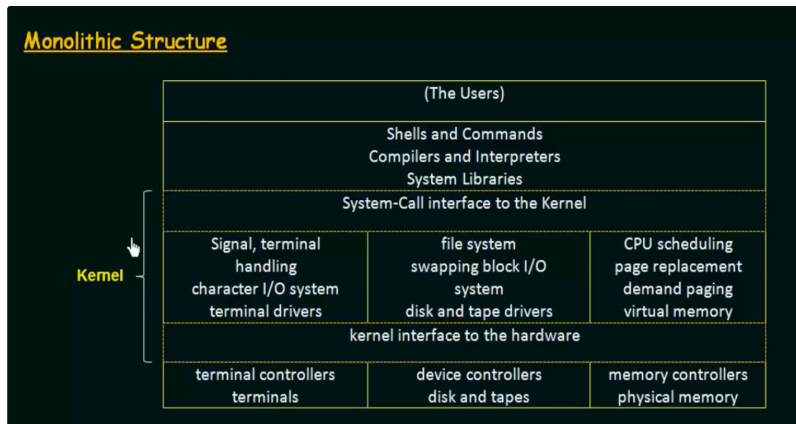


Structure of the Operating System



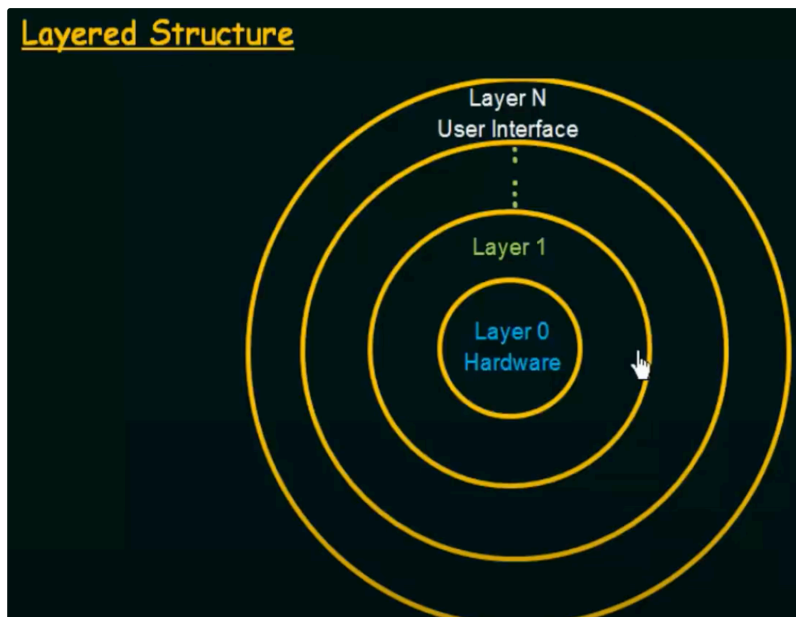
Monolithic Structure

In a **monolithic operating system**, many services and functions are bundled into a single large kernel. This can cause the “1 affects n” phenomenon—one failure can affect many components.



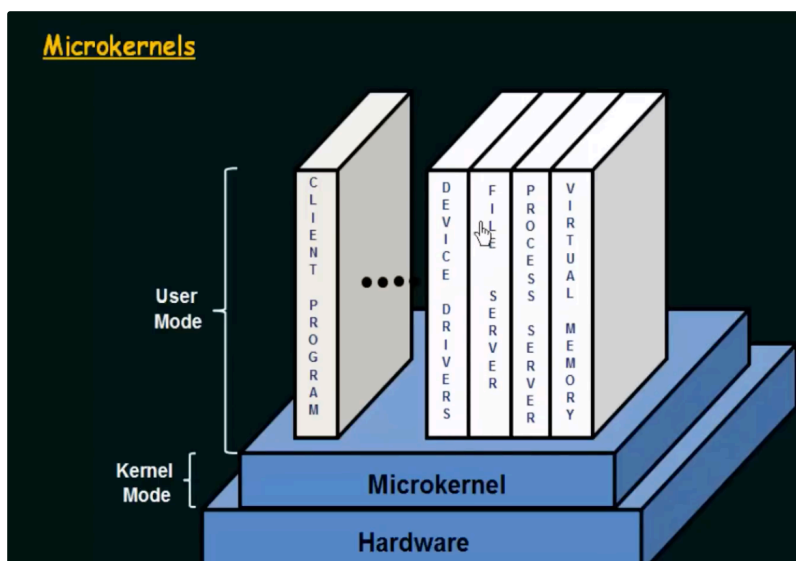
Layered Structure

A **layered OS structure** separates functionality into layers. Each layer has different responsibilities, making the design easier to build and maintain.



Microkernels

Microkernel architecture keeps only essential functions (like memory and process management) inside the kernel. Other services run in user space for improved reliability and security.



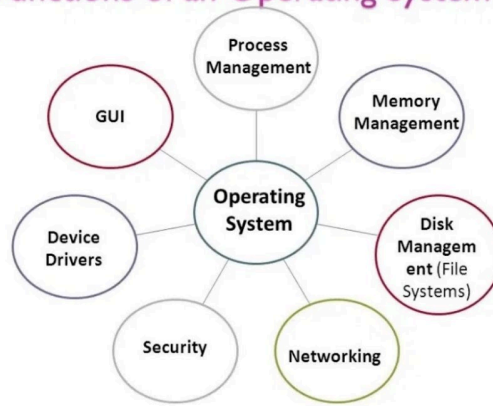
Primary Functions of an Operating System

An OS performs several essential functions, including:

- **Process Management:** Scheduling and managing active processes.
- **Memory Management:** Allocating and organizing memory use.
- **File System Management:** Managing file storage, access, and permissions.
- **Device Management:** Controlling hardware through drivers.
- **Security & Privacy:** Protecting data and managing access control.

Major Functions of the OS

Functions of an Operating System



- **User Interface:** Provides GUI or CLI for user interaction.
- **Resource Management:** Controls CPU, memory, and I/O devices.
- **File Management:** Handles storage, retrieval, and organization of files.
- **Task Management:** Manages execution of multiple processes.
- **Security Management:** Controls access and protects system resources.

Software Development

Definition

Software Development: The process of creating, designing, deploying, and supporting **software applications**.

Stages of Software Development Life Cycle (SDLC)

1. **Requirement Analysis:** Gathering and analyzing requirements from stakeholders.
2. **Design:** Planning the software architecture and design.
3. **Implementation:** Writing and compiling the source code.
4. **Testing:** Testing the software for defects and issues.
5. **Deployment:** Releasing the software for use.
6. **Maintenance:** Updating and maintaining the software post-deployment.

Models of Software Development

- **Waterfall Model:** Sequential design process, one phase after another.
- **Agile Model:** Iterative and incremental process with flexibility and customer feedback.
- **DevOps:** Integration of development and operations for continuous delivery.
- **Spiral Model:** Risk-driven iterative process combining Agile and Waterfall aspects.
- **Prototyping:** Developing preliminary versions to refine requirements and functionality.

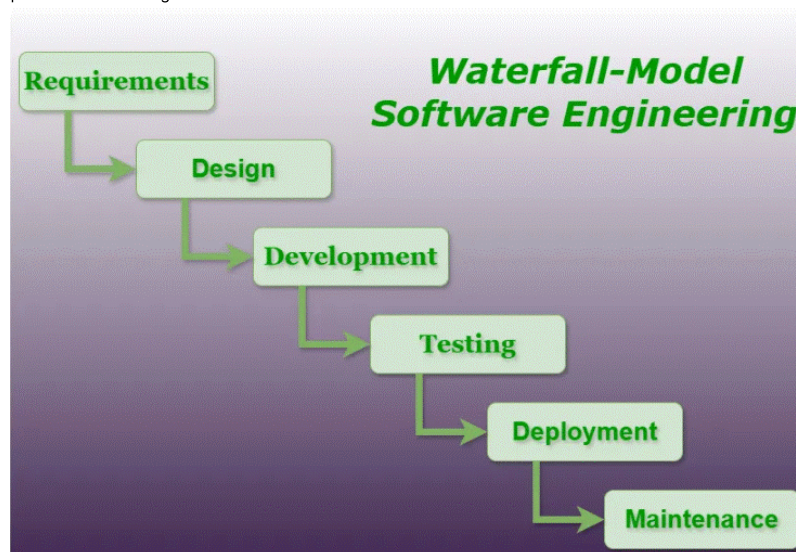
Preferred Models in the Digital Transformation Era

- **Agile:** Flexible, customer-centric, enables rapid delivery of software updates.
- **DevOps:** Promotes collaboration, automation, and continuous integration/delivery.

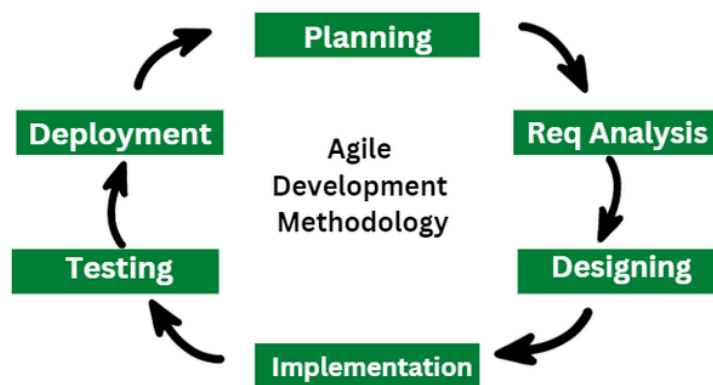
Software Development Models Overview

Definitions

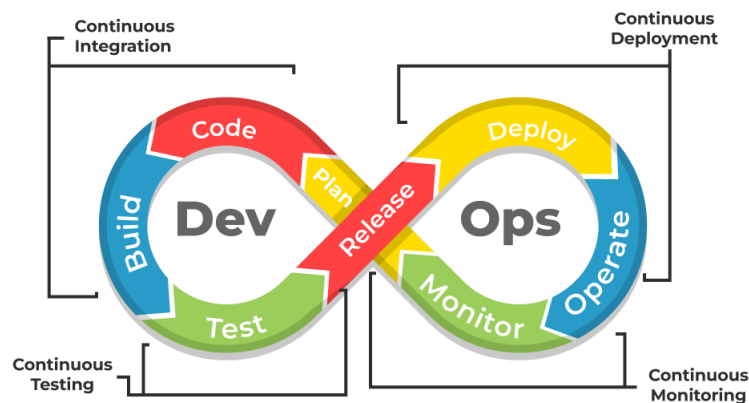
- **Waterfall Model:** A linear, sequential software development process where each phase (requirements, design, implementation, testing, deployment, maintenance) must be completed before moving to the next.



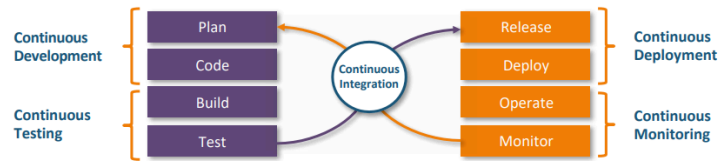
- **Agile Model:** An iterative and incremental approach focused on flexibility, customer collaboration, continuous delivery, and rapid feedback through sprints.



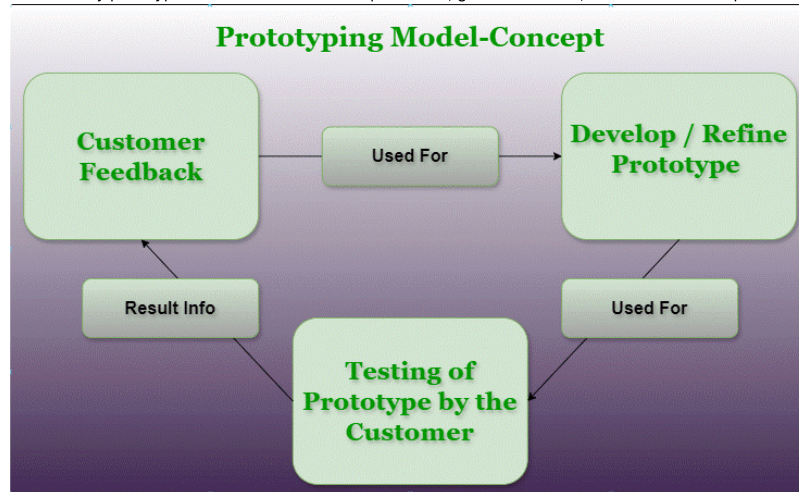
- **DevOps:** A set of practices combining development and IT operations, using automation and CI/CD pipelines to shorten delivery time and increase software reliability.



- **Spiral Model:** A risk-driven model combining Waterfall structure with iterative cycles. Each spiral involves planning, risk analysis, engineering, and evaluation.



- **Prototyping:** A model where an early prototype is built to understand requirements, gather feedback, and refine the final product's functionality.



Pros

- **Waterfall:** Simple, structured, and good for fixed requirements.
- **Agile:** Highly adaptable, frequent feedback, continuous improvement.
- **DevOps:** Faster delivery, automation efficiency, strong Dev-Ops collaboration.
- **Spiral:** Strong focus on risk management, flexible, good for complex projects.
- **Prototyping:** Helps refine requirements, early user feedback, reduces misunderstandings.

Cons

- **Waterfall:** Inflexible, costly if issues are found late, poor adaptability.
- **Agile:** Requires continuous involvement, unpredictable timelines, less documentation.
- **DevOps:** Cultural shift required, complex setup, potential security risks.
- **Spiral:** Expensive, complex to manage, not ideal for small projects.
- **Prototyping:** Risk of scope creep, mistaken for the final product, can be time-consuming.

Examples

- **Waterfall:** Regulatory software, fixed-requirement systems.
- **Agile:** Web/mobile apps, startup projects, customer-focused apps.
- **DevOps:** Cloud services, enterprise systems, CI/CD workflows.
- **Spiral:** Large complex systems, R&D projects, evolving-requirement systems.
- **Prototyping:** User interface design, educational tools, interactive applications.

Models of Software Development: Pros and Cons

Model	Description	Pros	Cons	Examples
Waterfall Model	Sequential design process (one phase after another).	- Simple and easy to understand and use. - Phases are clearly defined and structured. - Works well for smaller projects with well-understood requirements.	- Inflexible to changes once a phase is completed. - High risk as issues discovered late in the process can be costly. - Poor adaptability to evolving requirements.	- Software development for regulatory environments. - Fixed requirement projects, like construction software.
Agile Model	Iterative and incremental process with flexibility and customer feedback.	- High adaptability to changing requirements. - Frequent customer feedback improves satisfaction. - Continuous improvement through iterations.	- Requires active user involvement and continuous feedback. - Can be difficult to predict timelines and budget. - Less documentation can lead to knowledge gaps.	- Web and mobile application development. - Startups with evolving requirements. - Customer-focused software projects.
DevOps	Integration of development and operations for continuous delivery.	- Faster delivery and continuous integration. - Automation improves efficiency. - Enhances collaboration between development and operations teams.	- Requires significant cultural shift and collaboration. - Complex setup and maintenance. - Security concerns due to frequent changes.	- Cloud-based applications. - Large-scale enterprises with frequent updates. - Continuous integration/continuous delivery (CI/CD) pipelines.
Spiral Model	Combines iterative development (like Agile) with the systematic aspects of the Waterfall model, focusing on risk.	- Strong focus on risk management. - Flexibility allows for changes and refinements.	- Can be complex to manage and requires significant expertise. - May be expensive due to ongoing risk analysis.	- Large, complex projects with significant risk. - Systems with unclear requirements.

Preferred Models in the Digital Transformation Era: Agile and DevOps

Agile Method

- **Why Preferred**
 - **Adaptability:** Agile is flexible and allows organizations to quickly respond to rapid technological and market changes.
 - **Customer-Centric:** Focuses on continuous feedback and iterative improvements to meet evolving customer expectations.
 - **Faster Time-to-Market:** Iterative releases ensure quicker delivery of updates and features in a fast-paced digital environment.

DevOps Method

- **Why Preferred**
 - **Continuous Integration & Deployment (CI/CD):** Enables fast, reliable delivery of new features—essential for digital transformation.
 - **Collaboration:** Breaks down silos between development and operations teams for seamless technology integration.
 - **Automation & Efficiency:** Automates repetitive tasks, reduces errors, and speeds up the entire delivery pipeline.

Why Agile & DevOps Are Preferred

1. **Speed & Responsiveness:** Support rapid development cycles, quick updates, and continuous feedback.
2. **Customer Focus:** Both models ensure fast, reliable delivery of customer-valued features.
3. **Scalability & Flexibility:** They adapt well to complex, large-scale digital transformation projects.
4. **Collaboration & Integration:** Agile promotes teamwork; DevOps integrates development and operations for cohesive project management.
5. **Continuous Improvement:** Both encourage ongoing refinement to keep up with evolving technologies.

Conclusion

- Agile and DevOps are preferred in the digital transformation era because they offer **adaptability, speed, customer focus, and continuous delivery of high-quality software**.
- Their emphasis on **collaboration, automation, and continuous improvement** aligns perfectly with the **rapid innovation demands** of modern digital initiatives.